

TREES MASTER REVISION HANDBOOK —

C++

For SDE Placement Interviews — Deep Intuition | Pattern Recognition | Interview Ready

How to use this: Read once slowly for understanding. Revisit Section 12 the morning of your interview. Every **10-Second Recall** is your flashcard.

```
// Standard TreeNode definition used throughout this handbook
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

SECTION 1 — Trees from First Principles

Why Trees Exist

Trees solve problems that flat structures cannot efficiently handle:

- **Hierarchical relationships** — file systems, org charts, JSON/DOM, expression parsers
- **Logarithmic search** — BST gives $O(\log n)$ search vs $O(n)$ for arrays
- **Recursive decomposition** — divide-and-conquer maps naturally to trees
- **Ordered structure without contiguous memory** — unlike arrays, trees grow dynamically

The Data Structure Relationship Map

Array	flat, indexed, $O(1)$ random access, $O(n)$ insert
Linked List	flat, sequential, $O(n)$ access, $O(1)$ insert at known node
Tree	hierarchical, $O(\log n)$ search (balanced), branching paths
Graph	general, unordered; trees are graphs with n nodes, $n-1$ edges, no cycles

A tree is a graph with exactly:

- n nodes, $n - 1$ edges, no cycles
- One designated root
- Exactly one path between any two nodes

Visual Mental Model

```

      1      Root (Depth 0, Level 1)
     / \
    2   3   Depth 1, Level 2
   /\  /\
  4 5 6 7   Depth 2, Level 3   LEAVES here
  
```

Term	Definition	In the diagram
Root	Top-most node, no parent	Node 1
Leaf	Node with no children	Nodes 4, 5, 6, 7
Height of node	Longest path DOWN to any leaf	height(1)=2, height(2)=1, height(4)=0
Depth of node	Distance from root	depth(1)=0, depth(4)=2
Level	Depth + 1	Root is Level 1
Subtree at node v	v + all descendants	Subtree at 2 = {2, 4, 5}
Ancestor of v	All nodes on path from root to v	Ancestors of 5 = {1, 2}
Descendant of v	All nodes in subtree of v	Descendants of 2 = {4, 5}

What You Should Mentally See

- 1. **Each node as a subproblem** — Can I solve for children first, then combine?
- 2. **The root as the “boss”** — It orchestrates results from its subtrees
- 3. **Leaves as base cases** — Recursion bottoms out here
- 4. **Paths as “decisions”** — Every root-to-leaf path is a sequence of left/right choices
- 5. **Subtrees as independent trees** — Any node can be treated as a mini root

The Core Mental Model: A tree problem = asking the root a question that the root can only answer after asking its children the same question.

SECTION 2 — Universal Tree Thinking Framework

The 3 Questions for Every Tree Problem

- 1. **What does my function return for a subtree?** (int? bool? pair? void?)
- 2. **What information flows UP from child parent?** (postorder pattern)
- 3. **What information flows DOWN from parent child?** (preorder pattern)

Information Flow Patterns

Flow Direction	When It Appears	Traversal Used	Example
Child Parent	Subtree aggregation	Postorder	Height, Diameter, LCA
Parent Child	Carry constraints	Preorder	Path Sum, Serialize
Sorted order	BST problems	Inorder	Kth Smallest, Validate BST
Level by level	Width, shortest path	BFS	Right View, Zigzag
Both directions	Per-node global answers	Two DFS	Rerooting DP

The Master Decision Framework

READ THE PROBLEM		
Does it process LEVELS, WIDTH, or SHORTEST PATH?		
Yes		BFS (queue)
No		
Is it a BST problem (sorted property needed)?		
Yes		Inorder / BST range trick
No		
Does it need subtree results BEFORE computing at a node?		
Yes		Postorder DFS
No		
Does it carry information FROM ROOT downward?		
Yes		Preorder DFS
No		
Does it need BEST result over ALL paths or ALL nodes?		
Yes		Tree DP (postorder + global)
No		
Does it answer MANY ancestor/LCA queries on large tree?		
Yes		Binary Lifting
No		
Does it need subtree as a contiguous range?		
Yes		Euler Tour + Segment Tree

Pattern Recognition Quick Table

If the problem says...	Think...
height, depth, balanced	Postorder DFS
root-to-leaf path / sum	Preorder + running sum

BST, sorted, Kth element	Inorder traversal
level order, right view, zigzag	BFS
diameter, max path sum	Postorder + global max
LCA, common ancestor	Postorder DFS or Binary Lifting
distance K from node	BFS + parent pointers
burn, spread, infect	BFS + parent pointers
ancestor query	Euler Tour
"best" across all nodes/paths	Tree DP
top view, bottom view, vertical order	BFS + column indexing
serialize/deserialize	Preorder + null markers

SECTION 3 — DFS Master Revision

The Universal DFS Template

```
// Universal DFS Template
// Time: O(n) — every node visited once
// Space: O(h) — call stack = tree height
//     O(log n) balanced, O(n) skewed

int DFS(TreeNode* node) {
    if (!node) return BASE_CASE;    // Always handle null first

    // Optional: Preorder work here

    int leftResult = DFS(node->left);
    int rightResult = DFS(node->right);

    // Postorder: combine and return
```

```
    return combine(leftResult, rightResult, node->val);  
}
```

Key Insight: The *ORDER* in which you process `node` relative to recursive calls determines the traversal type.

Preorder DFS (Root Left Right)

Why it works: Process root BEFORE children. Root “knows” its context before exploring.

Use cases: Serialize tree, print root-to-leaf paths, copy tree, carry info FROM root TO leaves.

```
// Algorithm: Preorder Traversal  
// Purpose: Process each node before its subtrees  
// Time: O(n) | Space: O(h)  
  
void preorder(TreeNode* node, vector<int>& result) {  
    if (!node) return;  
  
    result.push_back(node->val); // Root FIRST  
    preorder(node->left, result);  
    preorder(node->right, result);  
}
```

Interview Insight

Any time you’re “carrying state downward” — pass it as a **parameter**. The parameter represents the accumulated state from root to current node.

Common Mistake

Forgetting to backtrack when building paths. Always `path.push_back()` before recursing and `path.pop_back()` after.

10-Second Recall

Root Left Right. “I decide BEFORE I explore.” Carry state as parameter.

Inorder DFS (Left Root Right)

Why BST loves inorder: For a BST, inorder produces elements in **sorted ascending order**.

```
BST:   5
      /\
     3 7   Inorder: 2  3  4  5  7   SORTED!
      /\
     2  4
```

Use cases: Kth smallest in BST, validate BST, BST Iterator, recover BST.

```
// Algorithm: Inorder Traversal
// Purpose:  Process each node between its subtrees (sorted for BST)
// Time:     O(n) | Space: O(h)

void inorder(TreeNode* node, vector<int>& result) {
    if (!node) return;

    inorder(node->left, result);
    result.push_back(node->val); // Root MIDDLE
    inorder(node->right, result);
}
```

Interview Insight

When you see a BST problem, ask: “Does inorder directly solve this?” Kth smallest, validation, and many others are trivial with inorder.

10-Second Recall

Left Root Right. BST = sorted order. Kth smallest = stop at Kth inorder node.

Postorder DFS (Left Right Root)

Why most hard problems use postorder: Children compute results and “report up” to parent. Parent combines and reports higher.

Mental model: Bottom-up aggregation.

Use cases: Height, diameter, balanced check, max path sum, LCA, delete a tree, tree DP.

```
// Algorithm: Postorder Traversal
// Purpose: Process each node AFTER both subtrees (bottom-up)
// Time: O(n) | Space: O(h)

int postorder(TreeNode* node) {
    if (!node) return BASE_CASE;

    int left = postorder(node->left);
    int right = postorder(node->right);

    return combine(left, right, node->val); // Root LAST
}
```

Interview Insight

If the problem asks for ANY subtree-level computation, it is secretly postorder. You cannot know the result at a node without knowing children's results first.

10-Second Recall

Left Right Root. Children report UP. Global variable for cross-subtree answers.

Iterative DFS (Explicit Stack)

When recursion depth is too large (10^5 nodes, skewed tree):

```
// Algorithm: Iterative Inorder
// Time: O(n) | Space: O(h)

vector<int> iterativeInorder(TreeNode* root) {
    stack<TreeNode*> st;
    TreeNode* curr = root;
    vector<int> result;

    while (curr || !st.empty()) {
```



```

while (curr) {      // Go as far left as possible
    st.push(curr);
    curr = curr->left;
}
curr = st.top(); st.pop();
result.push_back(curr->val); // Process
curr = curr->right;      // Move to right subtree
}
return result;
}

```

```

// Algorithm: Iterative Preorder
// Time: O(n) | Space: O(h)

vector<int> iterativePreorder(TreeNode* root) {
    if (!root) return {};
    stack<TreeNode*> st;
    st.push(root);
    vector<int> result;

    while (!st.empty()) {
        TreeNode* node = st.top(); st.pop();
        result.push_back(node->val); // Process immediately

        if (node->right) st.push(node->right); // Right first (LIFO)
        if (node->left) st.push(node->left);
    }
    return result;
}

```

SECTION 4 — BFS Master Revision

Level Order Traversal

Queue intuition: FIFO property ensures all of level L is processed before level L+1.

`levelSize = q.size()` at the loop start captures exactly the current level count.

Why BFS gives shortest paths: Explores nodes in increasing order of distance from source. First reach = minimum steps.

```
// Algorithm: Level Order Traversal
// Purpose: Process nodes level by level
// Time: O(n) | Space: O(w) w = max level width

vector<vector<int>> levelOrder(TreeNode* root) {
    if (!root) return {};
    queue<TreeNode*> q;
    q.push(root);
    vector<vector<int>> result;

    while (!q.empty()) {
        int levelSize = q.size();    // Snapshot current level count
        vector<int> currentLevel;

        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front(); q.pop();
            currentLevel.push_back(node->val);

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        result.push_back(currentLevel);
    }
    return result;
}
```

Key pattern: `int levelSize = q.size()` at the START of while loop. Loop exactly `levelSize` times to process one complete level.

Zigzag Level Order

```
// Algorithm: Zigzag Level Order
// Purpose: Alternate L R and R L each level
// Time: O(n) | Space: O(w)
```

```

vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
    if (!root) return {};
    queue<TreeNode*> q;
    q.push(root);
    bool leftToRight = true;
    vector<vector<int>> result;

    while (!q.empty()) {
        int size = q.size();
        vector<int> level(size);

        for (int i = 0; i < size; i++) {
            TreeNode* node = q.front(); q.pop();

            // Direction-aware insertion
            int pos = leftToRight ? i : (size - 1 - i);
            level[pos] = node->val;

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        leftToRight = !leftToRight;
        result.push_back(level);
    }
    return result;
}

```

10-Second Recall

Same as level order. Index `i` when L R, index `size-1-i` when R L. Flip flag each level.

Right Side View

```

// Algorithm: Right Side View
// Purpose: Last (rightmost) visible node at each level
// Time: O(n) | Space: O(w)

vector<int> rightSideView(TreeNode* root) {

```

```

if (!root) return {};
queue<TreeNode*> q;
q.push(root);
vector<int> result;

while (!q.empty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
        TreeNode* node = q.front(); q.pop();

        if (i == size - 1) // Last node in level
            result.push_back(node->val);

        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
}
return result;
}

```

Interview Insight

Right side view $\neq$ "just right children." If left subtree is deeper, left subtree nodes appear in result.

Level Averages

```

// Time: O(n) | Space: O(w)

vector<double> levelAverages(TreeNode* root) {
    if (!root) return {};
    queue<TreeNode*> q;
    q.push(root);
    vector<double> result;

    while (!q.empty()) {
        int size = q.size();
        double sum = 0;

```

```

    for (int i = 0; i < size; i++) {
        TreeNode* node = q.front(); q.pop();
        sum += node->val;
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    result.push_back(sum / size);
}
return result;
}

```

Vertical Order Traversal

Core idea: Assign each node a column index (root=0, left child=col-1, right child=col+1).
 BFS with (node, col, row) tuples. Sort within each column by (row, val) .

```

// Algorithm: Vertical Order Traversal
// Purpose: Group nodes by column; within column sort by (row, val)
// Time: O(n log n) | Space: O(n)

vector<vector<int>> verticalTraversal(TreeNode* root) {
    if (!root) return {};
    // map: col  vector of (row, val)
    map<int, vector<pair<int,int>>> colMap;
    // queue: (node, col, row)
    queue<tuple<TreeNode*, int, int>> q;
    q.push({root, 0, 0});

    while (!q.empty()) {
        auto [node, col, row] = q.front(); q.pop();
        colMap[col].push_back({row, node->val});

        if (node->left) q.push({node->left, col - 1, row + 1});
        if (node->right) q.push({node->right, col + 1, row + 1});
    }

    vector<vector<int>> result;
    for (auto& [col, nodes] : colMap) {
        sort(nodes.begin(), nodes.end()); // sort by (row, val)
    }
}

```

```
vector<int> colVals;  
for (auto& [row, val] : nodes) colVals.push_back(val);  
result.push_back(colVals);  
}  
return result;  
}
```

SECTION 5 — Binary Tree Patterns

HEIGHT OF TREE

Recognition signal: “Maximum depth”, “height”, “longest root-to-leaf path”

Intuition: Ask left child its height, ask right child its height, take max and add 1. Pure postorder.

```
// Algorithm: Tree Height  
// Purpose: Longest root-to-leaf path length  
// Time: O(n) | Space: O(h)  
  
int height(TreeNode* node) {  
    if (!node) return 0; // Base: null has height 0  
    return 1 + max(height(node->left), height(node->right));  
}
```

Interview Insight

Height of null=0. Height of leaf=1. Maximum depth = height of root.

Common Mistake

Off-by-one: some problems count edges (leaf=0), others count nodes (leaf=1). Clarify upfront.

10-Second Recall

```
return 1 + max(left, right) , base null 0
```

DIAMETER OF TREE

Recognition signal: “Longest path between ANY two nodes”, “diameter”

Intuition: Diameter path doesn't need to pass through root. At every node, diameter through it = `left_height + right_height` . Track global maximum across all nodes.

```
// Algorithm: Diameter of Binary Tree
// Purpose: Longest path (edges) between any two nodes
// Time: O(n) | Space: O(h)

class Solution {
    int maxDiam = 0;

    int helper(TreeNode* node) {
        if (!node) return 0;
        int left = helper(node->left);
        int right = helper(node->right);
        maxDiam = max(maxDiam, left + right); // Update global at each node
        return 1 + max(left, right); // Report height to parent
    }

public:
    int diameterOfBinaryTree(TreeNode* root) {
        helper(root);
        return maxDiam;
    }
};
```

Interview Insight

Diameter = globally maximum `(left_height + right_height)` at every node. Function returns **height** to parent but updates **diameter** as a side effect.

Common Mistake

Only checking path through root. Diameter can lie entirely within a subtree.

10-Second Recall

Postorder. Update global with `left + right`. Return `1 + max(left, right)`.

BALANCED BINARY TREE

Recognition signal: “Check if height-balanced” — no subtree heights differ by more than 1.

Intuition: Postorder. Use -1 as sentinel for “unbalanced” — propagates failure upward in one pass.

```
// Algorithm: Check Balanced (O(n) one-pass)
// Purpose: Verify  $|\text{height}(L) - \text{height}(R)| \leq 1$  for every node
// Time:  $O(n)$  | Space:  $O(h)$ 

int checkHeight(TreeNode* node) {
    if (!node) return 0;

    int left = checkHeight(node->left);
    if (left == -1) return -1; // Propagate failure immediately

    int right = checkHeight(node->right);
    if (right == -1) return -1;

    if (abs(left - right) > 1) return -1; // This node is unbalanced

    return 1 + max(left, right); // Return actual height
}

bool isBalanced(TreeNode* root) {
    return checkHeight(root) != -1;
}
```

Common Mistake

$O(n^2)$ naive: calling `height()` separately for each node. The -1 sentinel gives $O(n)$ in one pass.

10-Second Recall

Return -1 if $|\text{left} - \text{right}| > 1$ (propagates upward). Return actual height otherwise.

MAXIMUM PATH SUM

Recognition signal: “Maximum sum path between any two nodes”

Intuition: At each node, best single-branch gain = $\text{node} \rightarrow \text{val} + \max(0, \max(\text{leftGain}, \text{rightGain}))$. Update global with the full path $\text{node} \rightarrow \text{val} + \text{leftGain} + \text{rightGain}$. Negative branches are ignored with $\max(0, \text{gain})$.

```
// Algorithm: Maximum Path Sum
// Purpose: Max sum path between any two nodes
// Time: O(n) | Space: O(h)

class Solution {
    int maxSum = INT_MIN;

    int maxGain(TreeNode* node) {
        if (!node) return 0;

        int leftGain = max(maxGain(node->left), 0); // Discard negatives
        int rightGain = max(maxGain(node->right), 0);

        // Best path passing THROUGH current node (can't extend upward)
        maxSum = max(maxSum, node->val + leftGain + rightGain);

        // Return best SINGLE branch to parent
        return node->val + max(leftGain, rightGain);
    }

public:
    int maxPathSum(TreeNode* root) {
        maxGain(root);
        return maxSum;
    }
};
```

Interview Insight

`max(..., 0)` ignores negative subtrees — key insight. A path can “restart” at any node.

Common Mistake

Passing `leftGain + rightGain` upward — creates a forked path (illegal). Only one direction can be passed up.

10-Second Recall

`max(0, L) + max(0, R) + val` update global. Return `val + max(0, max(L, R))` .

PATH SUM PROBLEMS

I. Root-to-Leaf — Does Path with Sum K Exist?

```
// Time: O(n) | Space: O(h)

bool hasPathSum(TreeNode* node, int target) {
    if (!node) return false;
    if (!node->left && !node->right) return node->val == target; // Leaf check

    int rem = target - node->val;
    return hasPathSum(node->left, rem) || hasPathSum(node->right, rem);
}
```

II. Root-to-Leaf — All Paths with Sum K

```
// Time: O(n^2) | Space: O(n) — O(n) paths each O(n) long

void pathSumHelper(TreeNode* node, int target,
    vector<int>& path, vector<vector<int>>& result) {
    if (!node) return;

    path.push_back(node->val);

    if (!node->left && !node->right && node->val == target)
```

```

        result.push_back(path);
    else {
        pathSumHelper(node->left, target - node->val, path, result);
        pathSumHelper(node->right, target - node->val, path, result);
    }
    path.pop_back(); // BACKTRACK
}

vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
    vector<vector<int>> result;
    vector<int> path;
    pathSumHelper(root, targetSum, path, result);
    return result;
}

```

III. Any Subpath — Count Paths with Sum K (LC 437)

Trick: Prefix sums, same as “subarray sum = k” on arrays.

```

// Time: O(n) | Space: O(n)

int helper(TreeNode* node, long long target,
            unordered_map<long long, int>& prefixCount, long long curSum) {
    if (!node) return 0;

    curSum += node->val;
    int count = prefixCount.count(curSum - target) ?
                prefixCount[curSum - target] : 0;

    prefixCount[curSum]++;
    count += helper(node->left, target, prefixCount, curSum);
    count += helper(node->right, target, prefixCount, curSum);
    prefixCount[curSum]--; // Backtrack (undo for sibling branch)

    return count;
}

int pathSumIII(TreeNode* root, int targetSum) {
    unordered_map<long long, int> prefixCount;
    prefixCount[0] = 1; // Empty path has prefix sum 0
}

```

```

return helper(root, (long long)targetSum, prefixCount, 0);
}

```

Recognition Signals

Variant	Pattern
Root-to-leaf, exists?	Preorder, carry remaining target
Root-to-leaf, all paths	Preorder + backtracking
Any path, count	Preorder + hashmap prefix sums
Any path, max sum	Postorder + global max

LOWEST COMMON ANCESTOR (LCA)

Recognition signal: “Find LCA of nodes p and q”

Intuition: LCA = deepest node where p and q are in different subtrees (or one IS that node). Return node if it's p or q. If both children return non-null, current node is the split point.

```

// Algorithm: LCA — Binary Tree
// Time: O(n) | Space: O(h)

TreeNode* lowestCommonAncestor(TreeNode* node, TreeNode* p, TreeNode* q) {
    if (!node || node == p || node == q) return node;

    TreeNode* left = lowestCommonAncestor(node->left, p, q);
    TreeNode* right = lowestCommonAncestor(node->right, p, q);

    if (left && right) return node; // p and q split here LCA
    return left ? left : right;    // Both found on one side
}

```

```

// Algorithm: LCA — BST (uses ordering)
// Time: O(h) | Space: O(h)

TreeNode* lowestCommonAncestorBST(TreeNode* node, TreeNode* p, TreeNode* q) {

```

```

    if (p->val < node->val && q->val < node->val)
        return lowestCommonAncestorBST(node->left, p, q); // Both left

    if (p->val > node->val && q->val > node->val)
        return lowestCommonAncestorBST(node->right, p, q); // Both right

    return node; // Split point = LCA
}

```

Interview Insight

LCA is postorder. "If both sides return non-null" is the key merge condition.

Common Mistake

Assuming both p and q exist in tree. If not guaranteed, add existence checks.

10-Second Recall

Return node if it's p or q. If both sides non-null this IS LCA. Else propagate non-null side.

SYMMETRIC TREE

```

// Time: O(n) | Space: O(h)

bool isMirror(TreeNode* left, TreeNode* right) {
    if (!left && !right) return true;
    if (!left || !right) return false;
    if (left->val != right->val) return false;

    return isMirror(left->left, right->right) // Outer pair
        && isMirror(left->right, right->left); // Inner pair
}

bool isSymmetric(TreeNode* root) {
    return isMirror(root->left, root->right);
}

```

SAME TREE

```
// Time: O(n) | Space: O(h)

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q) return false;
    if (p->val != q->val) return false;
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}
```

INVERT BINARY TREE

```
// Algorithm: Invert Binary Tree
// Purpose: Swap all left and right children recursively
// Time: O(n) | Space: O(h)

TreeNode* invertTree(TreeNode* node) {
    if (!node) return nullptr;

    swap(node->left, node->right); // Swap children

    invertTree(node->left);
    invertTree(node->right);

    return node;
}
```

BOUNDARY TRAVERSAL

Strategy: Three separate passes — Left boundary (top → bottom, no leaf), All leaves (L → R), Right boundary (bottom → top, no leaf).

```
// Time: O(n) | Space: O(n)

bool isLeaf(TreeNode* node) {
```

```

    return !node->left && !node->right;
}

void addLeftBoundary(TreeNode* node, vector<int>& res) {
    if (!node || isLeaf(node)) return;
    res.push_back(node->val);
    if (node->left) addLeftBoundary(node->left, res);
    else          addLeftBoundary(node->right, res);
}

void addRightBoundary(TreeNode* node, vector<int>& res) {
    if (!node || isLeaf(node)) return;
    if (node->right) addRightBoundary(node->right, res);
    else          addRightBoundary(node->left, res);
    res.push_back(node->val);    // Add AFTER recursion (bottom-to-top)
}

void addLeaves(TreeNode* node, vector<int>& res) {
    if (!node) return;
    if (isLeaf(node)) { res.push_back(node->val); return; }
    addLeaves(node->left, res);
    addLeaves(node->right, res);
}

vector<int> boundaryTraversal(TreeNode* root) {
    if (!root) return {};
    vector<int> result;
    result.push_back(root->val);
    addLeftBoundary(root->left, result);
    addLeaves(root->left, result);
    addLeaves(root->right, result);
    addRightBoundary(root->right, result);
    return result;
}

```

TOP VIEW

Intuition: BFS + column indexing. For each column, record the FIRST node (BFS = top-to-bottom, so first = topmost).

```
// Time: O(n log n) | Space: O(n)

vector<int> topView(TreeNode* root) {
    if (!root) return {};
    map<int, int> colMap;           // col    first node value
    queue<pair<TreeNode*, int>> q;
    q.push({root, 0});

    while (!q.empty()) {
        auto [node, col] = q.front(); q.pop();

        if (colMap.find(col) == colMap.end()) // Only record FIRST
            colMap[col] = node->val;

        if (node->left) q.push({node->left, col - 1});
        if (node->right) q.push({node->right, col + 1});
    }

    vector<int> result;
    for (auto& [col, val] : colMap) result.push_back(val);
    return result;
}
```

BOTTOM VIEW

Intuition: Same as top view but always overwrite — last written in BFS = bottommost.

```
// Time: O(n log n) | Space: O(n)

vector<int> bottomView(TreeNode* root) {
    if (!root) return {};
    map<int, int> colMap;
    queue<pair<TreeNode*, int>> q;
    q.push({root, 0});

    while (!q.empty()) {
        auto [node, col] = q.front(); q.pop();
        colMap[col] = node->val;           // Always overwrite    last = bottom
    }
}
```



```

        if (node->left) q.push({node->left, col - 1});
        if (node->right) q.push({node->right, col + 1});
    }

    vector<int> result;
    for (auto& [col, val] : colMap) result.push_back(val);
    return result;
}

```

MAXIMUM WIDTH OF BINARY TREE

Intuition: Number nodes like a heap (root=0, left=2i, right=2i+1). Width = last_idx - first_idx + 1. Normalize per level to prevent overflow.

```

// Algorithm: Maximum Width
// Time: O(n) | Space: O(w)

int widthOfBinaryTree(TreeNode* root) {
    if (!root) return 0;
    queue<pair<TreeNode*, long long>> q;
    q.push({root, 0});
    int maxWidth = 0;

    while (!q.empty()) {
        int size = q.size();
        long long firstIdx = q.front().second; // For normalization
        long long lastIdx = firstIdx;

        for (int i = 0; i < size; i++) {
            auto [node, idx] = q.front(); q.pop();
            long long normIdx = idx - firstIdx; // Prevent overflow
            lastIdx = normIdx;

            if (node->left) q.push({node->left, 2 * normIdx});
            if (node->right) q.push({node->right, 2 * normIdx + 1});
        }
        maxWidth = max(maxWidth, (int)(lastIdx + 1));
    }
}

```

```
    return maxWidth;
}
```

Common Mistake

Not normalizing indices — without it, `2*idx` overflows for deep trees even with `long`.

DISTANCE K FROM NODE

Intuition: K steps can go DOWN (through children) or UP (through parent). Build parent map first, then BFS from target in all 3 directions.

```
// Algorithm: Nodes at Distance K
// Time:    O(n) | Space: O(n)

void buildParentMap(TreeNode* node, TreeNode* parent,
                    unordered_map<TreeNode*, TreeNode*>& parentMap) {
    if (!node) return;
    parentMap[node] = parent;
    buildParentMap(node->left, node, parentMap);
    buildParentMap(node->right, node, parentMap);
}

vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
    unordered_map<TreeNode*, TreeNode*> parentMap;
    buildParentMap(root, nullptr, parentMap);

    unordered_set<TreeNode*> visited;
    queue<TreeNode*> q;
    q.push(target);
    visited.insert(target);
    int distance = 0;

    while (!q.empty()) {
        if (distance == k) {
            vector<int> result;
            while (!q.empty()) {
                result.push_back(q.front()->val);
            }
        }
        int size = q.size();
        while (size-- > 0) {
            TreeNode* node = q.front(); q.pop();
            if (node->left && !visited.count(node->left)) {
                q.push(node->left);
                visited.insert(node->left);
            }
            if (node->right && !visited.count(node->right)) {
                q.push(node->right);
                visited.insert(node->right);
            }
            if (parentMap.count(node) && !visited.count(parentMap[node])) {
                q.push(parentMap[node]);
                visited.insert(parentMap[node]);
            }
        }
        distance++;
    }
    return result;
}
```

```

        q.pop();
    }
    return result;
}

int sz = q.size();
for (int i = 0; i < sz; i++) {
    TreeNode* node = q.front(); q.pop();

    // 3 directions: left child, right child, parent
    for (TreeNode* nb : {node->left, node->right, parentMap[node]}) {
        if (nb && !visited.count(nb)) {
            visited.insert(nb);
            q.push(nb);
        }
    }
}
distance++;
}
return {};
}

```

Recognition Signal

“Distance K from a specific node” BFS + parent pointers. Never pure DFS (can’t go up).

BURN TREE PROBLEM

Intuition: Identical to Distance K. Fire spreads in all 3 directions simultaneously. Count BFS levels until all nodes are visited.

```

// Algorithm: Time to Burn Tree
// Time:    O(n) | Space: O(n)

TreeNode* findNode(TreeNode* root, int target) {
    if (!root) return nullptr;
    if (root->val == target) return root;
    TreeNode* left = findNode(root->left, target);
    return left ? left : findNode(root->right, target);
}

```

```

}

int timeToBurnTree(TreeNode* root, int startVal) {
    unordered_map<TreeNode*, TreeNode*> parentMap;
    buildParentMap(root, nullptr, parentMap); // Reuse from above

    TreeNode* start = findNode(root, startVal);
    unordered_set<TreeNode*> visited;
    queue<TreeNode*> q;
    q.push(start);
    visited.insert(start);
    int time = -1;

    while (!q.empty()) {
        time++;
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            for (TreeNode* nb : {node->left, node->right, parentMap[node]}) {
                if (nb && !visited.count(nb)) {
                    visited.insert(nb);
                    q.push(nb);
                }
            }
        }
    }
    return time;
}

```

10-Second Recall

Burn Tree = Distance K but count all levels till BFS ends. Same parent map + BFS pattern.

SECTION 6 — Binary Search Trees (BST)

BST Core Properties

For every node N:

ALL values in N->left subtree < N->val

ALL values in N->right subtree > N->val

- Inorder traversal = **sorted ascending sequence**
 - Average: $O(\log n)$ for search, insert, delete
 - Worst case (fully skewed): $O(n)$
 - Balanced BSTs (AVL, Red-Black): guaranteed $O(\log n)$
-

SEARCH IN BST

// Time: $O(h)$ | Space: $O(h)$ recursive, $O(1)$ iterative

```
TreeNode* searchBST(TreeNode* node, int target) {
    if (!node) return nullptr;
    if (node->val == target) return node;
    if (target < node->val) return searchBST(node->left, target);
    return searchBST(node->right, target);
}
```

INSERT IN BST

// Time: $O(h)$ | Space: $O(h)$

```
TreeNode* insertIntoBST(TreeNode* node, int val) {
    if (!node) return new TreeNode(val); // Create leaf here

    if (val < node->val)
        node->left = insertIntoBST(node->left, val);
    else if (val > node->val)
        node->right = insertIntoBST(node->right, val);
    // If val == node->val: handle duplicates per problem spec

    return node;
}
```

DELETE IN BST

Three cases: (1) Leaf remove. (2) One child replace with child. (3) Two children replace with inorder successor, delete successor.

```
// Time: O(h) | Space: O(h)
```

```
TreeNode* findMin(TreeNode* node) {  
    while (node->left) node = node->left;  
    return node;  
}
```

```
TreeNode* deleteNode(TreeNode* node, int key) {  
    if (!node) return nullptr;  
  
    if (key < node->val)  
        node->left = deleteNode(node->left, key);  
    else if (key > node->val)  
        node->right = deleteNode(node->right, key);  
    else {  
        // Found the node to delete  
        if (!node->left) return node->right; // Case 1 & 2  
        if (!node->right) return node->left; // Case 2  
  
        // Case 3: Two children find inorder successor  
        TreeNode* successor = findMin(node->right);  
        node->val = successor->val; // Replace value  
        node->right = deleteNode(node->right, successor->val); // Delete successor  
    }  
    return node;  
}
```

Common Mistake

Forgetting Case 3. Key insight: replace the **value** (not the node) with successor's value, then delete successor.

VALIDATE BST

Wrong approach: Checking only immediate parent-child misses global violations.

```
// Algorithm: Validate BST
// Purpose: Verify BST property holds globally
// Time: O(n) | Space: O(h)

bool validate(TreeNode* node, long long minVal, long long maxVal) {
    if (!node) return true;

    if (node->val <= minVal || node->val >= maxVal)
        return false; // Outside valid range

    return validate(node->left, minVal, node->val) // Left: tighten max
        && validate(node->right, node->val, maxVal); // Right: tighten min
}

bool isValidBST(TreeNode* root) {
    return validate(root, LLONG_MIN, LLONG_MAX);
}
```

Common Mistake

```
// WRONG: Only checks immediate parent
if (node->left && node->left->val >= node->val) return false;

// CORRECT: Pass (min, max) bounds from root downward
```

10-Second Recall

`validate(node, min, max)` . Left tightens upper bound. Right tightens lower bound.

KTH SMALLEST IN BST

Intuition: Inorder = sorted. Kth inorder element = Kth smallest.

```
// Time: O(h + k) | Space: O(h)

class Solution {
    int count = 0, result = -1;

    void inorder(TreeNode* node, int k) {
        if (!node || count >= k) return;
        inorder(node->left, k);
        if (++count == k) { result = node->val; return; }
        inorder(node->right, k);
    }

public:
    int kthSmallest(TreeNode* root, int k) {
        inorder(root, k);
        return result;
    }
};
```

Kth Largest: Reverse inorder (Right → Root → Left), count k.

```
void reverseInorder(TreeNode* node, int k, int& count, int& result) {
    if (!node || count >= k) return;
    reverseInorder(node->right, k, count, result);
    if (++count == k) { result = node->val; return; }
    reverseInorder(node->left, k, count, result);
}
```

INORDER SUCCESSOR

Definition: Smallest node with value GREATER than p.

```
// Algorithm: Inorder Successor in BST
// Time: O(h) | Space: O(1)

TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {
    TreeNode* successor = nullptr;
```



```

while (root) {
    if (p->val < root->val) {
        successor = root;    // Potential successor (went left here)
        root = root->left;
    } else {
        root = root->right;    // p is to the right, go right
    }
}
return successor;
}

```

INORDER PREDECESSOR

Definition: Largest node with value LESS than p.

```

// Time: O(h) | Space: O(1)

TreeNode* inorderPredecessor(TreeNode* root, TreeNode* p) {
    TreeNode* predecessor = nullptr;

    while (root) {
        if (p->val > root->val) {
            predecessor = root;    // Potential predecessor (went right here)
            root = root->right;
        } else {
            root = root->left;
        }
    }
    return predecessor;
}

```

RECOVER BST

Intuition: Inorder should be sorted. Two swapped nodes create violations. First violation: `first = prev` . Second violation: always update `second = curr` .

```
// Time: O(n) | Space: O(h)
```

```
class Solution {
    TreeNode *prev = nullptr, *first = nullptr, *second = nullptr;

    void inorder(TreeNode* node) {
        if (!node) return;
        inorder(node->left);

        if (prev && prev->val > node->val) {
            if (!first) first = prev; // First violation: capture the larger node
            second = node;           // Always update second (smaller node)
        }
        prev = node;

        inorder(node->right);
    }

public:
    void recoverTree(TreeNode* root) {
        inorder(root);
        swap(first->val, second->val);
    }
};
```

Interview Insight

Works for both adjacent and non-adjacent swaps. For adjacent: one violation, `first=prev` , `second=node` at same point. For non-adjacent: two violations.

BST ITERATOR

Intuition: Simulate inorder lazily with an explicit stack. Amortized $O(1)$ per `next()` call.

```
// Time: O(1) amortized per next() | Space: O(h)
```

```
class BSTIterator {
    stack<TreeNode*> st;
```

```

void pushAllLeft(TreeNode* node) {
    while (node) {
        st.push(node);
        node = node->left;
    }
}

public:
    BSTIterator(TreeNode* root) {
        pushAllLeft(root);
    }

    int next() {
        TreeNode* node = st.top(); st.pop();
        pushAllLeft(node->right); // Push right subtree's leftmost path
        return node->val;
    }

    bool hasNext() {
        return !st.empty();
    }
};

```

SECTION 7 — Advanced Tree Patterns

TREE DP

What It Is

Tree DP = postorder DFS where the **return value IS the DP state**. Each node computes a state from its children's states.

State Definition Process

1. "What information does a subtree need to give its parent?" Define return type

2. "How do I combine children's states?" Recurrence
 3. "What is the state for null?" Base case
-

Example: House Robber III

State: Return {robThis, skipThis} where:

- robThis = max if we rob current node (forces children skipped)
- skipThis = max if we skip current node (children free)

```
// Algorithm: House Robber III
// Purpose: Maximum non-adjacent value sum on tree
// Time: O(n) | Space: O(h)

pair<int,int> rob(TreeNode* node) {
    if (!node) return {0, 0};

    auto [leftRob, leftSkip] = rob(node->left);
    auto [rightRob, rightSkip] = rob(node->right);

    // Rob this: children MUST be skipped
    int robThis = node->val + leftSkip + rightSkip;

    // Skip this: children can be robbed OR skipped
    int skipThis = max(leftRob, leftSkip) + max(rightRob, rightSkip);

    return {robThis, skipThis};
}

int houseRobber(TreeNode* root) {
    auto [robRoot, skipRoot] = rob(root);
    return max(robRoot, skipRoot);
}
```

Tree DP General Pattern

```
// Tree DP Template
ReturnType treeDP(TreeNode* node) {
    if (!node) return BASE_STATE;
```

```

    ReturnType leftState = treeDP(node->left);
    ReturnType rightState = treeDP(node->right);

    ReturnType currentState = recurrence(leftState, rightState, node->val);
    updateGlobalAnswer(currentState);

    return currentState;
}

```

REROOTING DP

What It Is

When the answer at each node requires knowing BOTH its subtree AND the rest of the tree. Naive: $O(n^2)$. Rerooting: $O(n)$ with two DFS passes.

Classic Problem: Sum of Distances in Tree (LC 834)

Rerooting formula: When moving root from parent P to child C:

- $sz[C]$ nodes get 1 closer subtract $sz[C]$
- $n - sz[C]$ nodes get 1 farther add $n - sz[C]$
- $dist[C] = dist[P] + n - 2 * sz[C]$

```

// Algorithm: Sum of Distances (Rerooting DP)
// Time: O(n) | Space: O(n)

class Solution {
    int n;
    vector<vector<int>> adj;
    vector<int> dist, sz;

    // First DFS: compute subtree sizes and dist from root
    void dfs1(int node, int parent) {
        sz[node] = 1;
        for (int v : adj[node]) {
            if (v == parent) continue;
            dfs1(v, node);
        }
    }
}

```

```

        sz[node] += sz[v];
        dist[node] += dist[v] + sz[v]; // All nodes in v's subtree are 1 step farther
    }
}

// Second DFS: reroot and propagate answer
void dfs2(int node, int parent) {
    for (int v : adj[node]) {
        if (v == parent) continue;
        // Moving root from node to v:
        dist[v] = dist[node] - sz[v] + (n - sz[v]);
        dfs2(v, node);
    }
}

public:
vector<int> sumOfDistancesInTree(int N, vector<vector<int>>& edges) {
    n = N;
    adj.assign(n, {});
    dist.assign(n, 0);
    sz.assign(n, 0);

    for (auto& e : edges) {
        adj[e[0]].push_back(e[1]);
        adj[e[1]].push_back(e[0]);
    }

    dfs1(0, -1);
    dfs2(0, -1);
    return dist;
}
};

```

Recognition Signal

“For every node, compute something based on distances to all other nodes” Rerooting DP.

BINARY LIFTING

What It Is

Sparse table for tree ancestors. $up[v][j]$ = 2^j -th ancestor of v . Supports $O(\log n)$ Kth ancestor and LCA queries.

```
// Algorithm: Binary Lifting
// Preprocessing:  $O(n \log n)$  Time,  $O(n \log n)$  Space
// Query:  $O(\log n)$  per LCA / Kth ancestor

const int LOG = 20; // Sufficient for  $n \leq 10^6$ 
const int MAXN = 100005;

int up[MAXN][LOG]; //  $up[v][j]$  =  $2^j$ -th ancestor of  $v$ 
int depth[MAXN];
vector<int> adj[MAXN];
int n;

// Preprocessing: DFS to fill up[][] table
void dfs(int node, int parent) {
    up[node][0] = parent;

    // Fill sparse table for powers of 2
    for (int j = 1; j < LOG; j++)
        up[node][j] = up[up[node][j-1]][j-1];

    for (int v : adj[node]) {
        if (v != parent) {
            depth[v] = depth[node] + 1;
            dfs(v, node);
        }
    }
}

void preprocess(int root) {
    depth[root] = 0;
    dfs(root, root); // Parent of root = root (sentinel)
}
```

```

// Kth Ancestor Query: O(log n)
int kthAncestor(int node, int k) {
    for (int j = 0; j < LOG; j++)
        if ((k >> j) & 1) // If j-th bit of k is set
            node = up[node][j];
    return node;
}

// LCA Query: O(log n)
int LCA(int u, int v) {
    // Step 1: Bring to same depth
    if (depth[u] < depth[v]) swap(u, v);
    u = kthAncestor(u, depth[u] - depth[v]); // Lift u up

    if (u == v) return u; // One was ancestor of other

    // Step 2: Binary lift both until one step below LCA
    for (int j = LOG - 1; j >= 0; j--)
        if (up[u][j] != up[v][j]) {
            u = up[u][j];
            v = up[v][j];
        }

    return up[u][0]; // Parent of current position = LCA
}

```

Recognition Signal

“Kth ancestor” OR “LCA on large tree” OR “many LCA queries” Binary Lifting.

10-Second Recall

$up[v][j] = up[up[v][j-1]][j-1]$. Kth ancestor = jump by set bits of k. LCA = equalize depth, then lift both until parents differ.

EULER TOUR

What It Is

DFS timestamp trick. Assign `tin` (entry time) and `tout` (exit time) to each node. **Subtree of v = contiguous segment $[tin[v], tout[v]]$** in the flat array.

Key Property

v is ancestor of u $tin[v] \leq tin[u]$ AND $tout[v] \geq tout[u]$

```
// Algorithm: Euler Tour
// Time:    O(n) preprocessing | O(1) ancestor query | Space: O(n)

const int MAXN = 100005;
int tin[MAXN], tout[MAXN], timer_val = 0;
vector<int> adj[MAXN];

void dfs(int node, int parent) {
    tin[node] = timer_val++;

    for (int v : adj[node])
        if (v != parent)
            dfs(v, node);

    tout[node] = timer_val++;
}

bool isAncestor(int v, int u) {
    return tin[v] <= tin[u] && tout[v] >= tout[u];
}
```

Applications

Application	How Euler Tour Helps
O(1) ancestor check	$tin[v] \leq tin[u] \leq tout[v]$
Subtree sum/max/min queries	Range $[tin[v], tout[v]]$ on segment tree
Subtree DP with updates	Flatten tree → 1D array, apply BIT/SegTree
Path queries	Use LCA to decompose path into two root-paths

Recognition Signal

“Multiple ancestor queries” OR “subtree range queries with updates” Euler Tour + Segment Tree.

SECTION 8 — Interview Patterns Cheat Sheet

Pattern	Recognition Signal	Solution Idea	Time	Space
DFS Preorder	Serialize, copy tree, root-to-leaf	Root L R; carry state as parameter	$O(n)$	$O(h)$
DFS Inorder	BST sorted order, Kth element	L Root R; BST = sorted	$O(n)$	$O(h)$
DFS Postorder	Height, diameter, LCA, subtree agg	L R Root; children report UP	$O(n)$	$O(h)$
BFS Level Order	Level processing, width, shortest path	Queue + size = q.size()	$O(n)$	$O(w)$
Tree DP	Best across all nodes/paths, house robber	Postorder, return DP state tuple	$O(n)$	$O(h)$
Binary Lifting	LCA large tree, Kth ancestor, many queries	Preprocess up[v][j] table	$O(n \log n)$	$O(n \log n)$
Euler Tour	Flatten tree, $O(1)$ ancestor, subtree range	tin/tout via DFS	$O(n)$	$O(n)$
Subtree Aggregation	Count/sum/max in subtree	Postorder, combine children	$O(n)$	$O(h)$
Root-to-Leaf Paths	Path exists, all paths, backtrack	Preorder + carry target	$O(n)$	$O(h)$
Diameter Style	“Any two nodes”, longest path	Postorder + global L + R	$O(n)$	$O(h)$

Distance K / Burn	All nodes K away, fire spread	Parent map + BFS	O(n)	O(n)
Column Indexing	Top/bottom/vertical view	BFS + col map, root=0 ±1	O(n log n)	O(n)
Rerooting	Every node as root, sum of distances	Two DFS: subtree then reroot	O(n)	O(n)
BST Property	Search, validate, Kth, successor	Use ordering; eliminate half	O(h)	O(h)

SECTION 9 — Most Important Trees Questions

Easy

#	Problem	Why It Matters	Pattern	Priority
1	Maximum Depth (LC 104)	Foundation of all height problems	Postorder	Must Do
2	Same Tree (LC 100)	Base for subtree checks	DFS comparison	Must Do
3	Symmetric Tree (LC 101)	Mirror DFS pattern	Two-pointer DFS	Must Do
4	Invert Binary Tree (LC 226)	Clean postorder swap	Postorder	Must Do
5	Path Sum (LC 112)	Root-to-leaf foundation	Preorder + target	Must Do
6	Search in BST (LC 700)	BST basics	BST search	Must Do
7	Insert into BST (LC 701)	BST insert pattern	BST insert	Must Do

8	Sorted Array to BST (LC 108)	Build balanced BST	Divide & conquer	
9	Merge Two Binary Trees (LC 617)	Simultaneous traversal	DFS	
10	Subtree of Another Tree (LC 572)	Pattern matching	DFS + isSameTree	

Medium

#	Problem	Why It Matters	Pattern	Priority
1	Level Order Traversal (LC 102)	BFS foundation	BFS	Must Do
2	Right Side View (LC 199)	BFS last per level	BFS	Must Do
3	Diameter (LC 543)	Classic Tree DP	Postorder + global	Must Do
4	Balanced Binary Tree (LC 110)	Height + -1 sentinel	Postorder	Must Do
5	Path Sum II (LC 113)	Backtracking on tree	Preorder + backtrack	Must Do
6	Path Sum III (LC 437)	Prefix sum on tree	Preorder + hashmap	Must Do
7	LCA Binary Tree (LC 236)	FAANG classic	Postorder	Must Do
8	Validate BST (LC 98)	BST core, common trap	Range validation	Must Do
9	Kth Smallest in BST (LC 230)	Inorder + counter	Inorder	Must Do
10	All Nodes Distance K (LC 863)	Parent map + BFS	BFS + parent	Must Do
11	Max Width (LC 662)	Index normalization	BFS + heap indices	Must Do

12	Construct from Pre+Inorder (LC 105)	Build tree	Divide + hashmap	Must Do
13	Zigzag Traversal (LC 103)	BFS direction toggle	BFS	
14	Top View / Bottom View	Column-first/last	BFS + colMap	Must Do
15	Recover BST (LC 99)	Inorder violation	Inorder + 2 ptrs	Pattern Builder
16	Serialize/Deserialize (LC 297)	System design trees	Preorder + markers	Pattern Builder
17	Flatten to Linked List (LC 114)	Preorder rearrangement	Preorder	
18	BST to Greater Sum (LC 1038)	Reverse inorder	Reverse inorder	
19	Vertical Traversal (LC 987)	Sort by (col,row,val)	BFS + colMap	
20	Next Right Pointers (LC 116)	Level linking	BFS or O(1)	

Hard

#	Problem	Why It Matters	Pattern	Priority
1	Max Path Sum (LC 124)	FAANG favorite	Postorder + global	Must Do
2	House Robber III (LC 337)	Tree DP with states	Postorder + pair	Must Do
3	Burn Tree (GFG/OA)	BFS + parent	Multi-source BFS	Must Do
4	Binary Tree Cameras (LC 968)	Greedy postorder	Postorder greedy	Pattern Builder
5	Count Complete Nodes (LC 222)	Binary search on tree	Height + bin search	Pattern Builder
6	Sum of Distances (LC 834)	Rerooting DP	Two-pass DFS	Pattern Builder

7	Morris Traversal (GFG)	O(1) space inorder	Thread-based	Pattern Builder
8	Binary Lifting LCA	Large tree ancestor	Sparse table	Pattern Builder
9	Boundary Traversal (GFG)	Three-part traversal	DFS composition	
10	Vertical Traversal (LC 987)	Stable sort required	BFS + sort	

SECTION 10 — Tree Problem Recognition Guide

The Mental Flowchart

READ THE PROBLEM

Keywords: level, width, shortest, right view, zigzag?

YES

BFS

NO

Keywords: BST, sorted, kth smallest/largest, validate?

YES

Inorder / BST range trick

NO

Need result from BELOW a node before processing it?

YES

Postorder DFS

NO

Carry info FROM ROOT downward to leaves?

YES

Preorder DFS

NO

Need BEST result over ALL paths or ALL nodes?

YES	Tree DP (global var)
NO	
Multiple LCA / ancestor queries on large tree?	
YES	Binary Lifting
NO	
Subtree queries with updates? $O(1)$ ancestor check?	
YES	Euler Tour + Seg Tree
NO	
Answer at each node depends on rest of tree too?	
YES	Rerooting DP (2 DFS)

Quick Recognition Keyword Table

Keyword in Problem	Immediate Pattern
"level", "level order"	BFS
"shortest path", "minimum steps"	BFS
"right side view", "left side view"	BFS, last/first per level
"zigzag", "spiral"	BFS + direction toggle
"height", "maximum depth"	Postorder DFS
"balanced"	Postorder, -1 sentinel
"diameter", "any two nodes"	Postorder + global max
"path sum", "root to leaf"	Preorder + carry target
"all root-to-leaf paths"	Preorder + backtracking
"path sum", "any path", "max"	Postorder Tree DP
"count subpaths"	Preorder + prefix sum hashmap
"sorted", "kth smallest/largest"	Inorder BST
"validate BST"	Range validation (min, max)

"LCA", "lowest common ancestor"	Postorder or Binary Lifting
"distance K from node"	Parent map + BFS
"burn", "spread", "infect"	Parent map + BFS
"ancestor query"	Euler Tour
"serialize", "deserialize"	Preorder + null markers
"construct from pre+in"	Divide + inorder hashmap
"house robber", "no adjacent"	Tree DP, pair state
"cameras", "minimize"	Greedy postorder
"top view", "bottom view"	BFS + column map
"vertical order"	BFS + (col, row, val) sort
"maximum width"	BFS + index normalization
"sum of distances"	Rerooting DP
"kth ancestor"	Binary Lifting

Mental Shortcuts

- **"Any two nodes"** global variable + postorder
- **"Distance K"** parent map + BFS (can path go UP? yes parent map)
- **BST problem** try inorder first — does sorted order solve it?
- **"For every node"** and answer needs rest of tree too Rerooting DP
- **Large n + LCA** Binary Lifting. Small n + one LCA simple postorder
- **Range DS on tree** Euler Tour to flatten, then Segment Tree / BIT

SECTION 11 — Complexity Revision Sheet

Algorithm / Problem	Time	Space	Key Note
DFS (any order)	$O(n)$	$O(h)$	$h=O(\log n)$ balanced, $O(n)$ skewed

BFS Level Order	$O(n)$	$O(w)$	w up to $O(n/2)$ for last level
Tree Height	$O(n)$	$O(h)$	
Diameter	$O(n)$	$O(h)$	Global max trick
Balanced Check	$O(n)$	$O(h)$	-1 sentinel, one pass
Maximum Path Sum	$O(n)$	$O(h)$	Global max, ignore negatives
Path Sum I (exists?)	$O(n)$	$O(h)$	Early exit possible
Path Sum II (all paths)	$O(n^2)$	$O(n)$	$O(n)$ paths $\times$ $O(n)$ copy
Path Sum III (any subpath)	$O(n)$	$O(n)$	Prefix sum + unordered_map
LCA — Binary Tree	$O(n)$	$O(h)$	Postorder
LCA — BST	$O(h)$	$O(h)$	$O(\log n)$ balanced
Binary Lifting (preprocess)	$O(n \log n)$	$O(n \log n)$	
Binary Lifting (query)	$O(\log n)$	—	Per query
Euler Tour (preprocess)	$O(n)$	$O(n)$	
Euler Tour (ancestor query)	$O(1)$	—	Per query
BST Search / Insert	$O(h)$	$O(h)$	$O(\log n)$ balanced
BST Delete	$O(h)$	$O(h)$	Inorder successor for 2-child
Validate BST	$O(n)$	$O(h)$	
Kth Smallest in BST	$O(h + k)$	$O(h)$	
Inorder Successor / Predecessor	$O(h)$	$O(1)$	Iterative
Recover BST	$O(n)$	$O(h)$	
BST Iterator	$O(1)$ amortized	$O(h)$	Lazy inorder
Vertical Order Traversal	$O(n \log n)$	$O(n)$	Sort by (col, row, val)
Top View / Bottom View	$O(n \log n)$	$O(n)$	Sort by column key

Maximum Width	$O(n)$	$O(w)$	Index normalization
Distance K from Node	$O(n)$	$O(n)$	Parent map + BFS
Burn Tree	$O(n)$	$O(n)$	Same as distance K
Boundary Traversal	$O(n)$	$O(h)$	Three traversals
Rerooting DP	$O(n)$	$O(n)$	Two DFS passes
House Robber III	$O(n)$	$O(h)$	Pair state return
Morris Traversal	$O(n)$	$O(1)$	Modifies tree temporarily
Serialize / Deserialize	$O(n)$	$O(n)$	Preorder + markers
Build from Pre + Inorder	$O(n)$	$O(n)$	Hashmap for $O(1)$ lookup
Count Complete Nodes	$O(\log^2 n)$	$O(\log n)$	Binary search on levels
Symmetric Tree	$O(n)$	$O(h)$	
Invert Tree	$O(n)$	$O(h)$	
Same Tree	$O(n)$	$O(h)$	
Tree DP (general)	$O(n)$	$O(h)$	State in return value

SECTION 12 — One-Day Placement

Revision Sheet

Read this the morning of your interview. 30 minutes is enough.

20 Most Important Ideas

1. **if (!node) return BASE** — Every recursive function starts here. Always.
2. **Postorder is king** — Most hard problems aggregate upward. Default to postorder.
3. **Inorder of BST = sorted array** — Any BST problem: ask “does inorder solve it?” first.
4. **BFS = level-by-level** — Queue + `int levelSize = q.size()` snapshot per iteration.

5. **Global variable trick** — Diameter, max path sum: update global inside recursive call. Return height/gain, not answer.
6. **Height of null = 0**, height of leaf = 1. Consistent. No off-by-one.
7. **Diameter = max(left + right)** at every node. Path need not go through root.
8. **BST validation needs (minVal, maxVal) bounds**, not just parent check. Use `LLONG_MIN/MAX`.
9. **LCA = first split point** — Both children return non-null — current node is LCA.
10. **Ignore negative paths with max(0, gain)** — Key for max path sum.
11. **BST delete 2-child** = replace with `findMin(right)` value, then delete that min from right.
12. **Distance K = parent map + BFS** — Build `unordered_map<TreeNode*, TreeNode*>` first.
13. **Binary Lifting** = $O(n \log n)$ preprocess, $O(\log n)$ per LCA query. For large trees.
14. **Euler Tour** — `tin/tout` gives $O(1)$ ancestor, subtree = contiguous range.
15. **Tree DP state = what the subtree returns** — Choose carefully; often a `pair<int,int>`.
16. **Rerooting** = two DFS. `dist[child] = dist[parent] + n - 2 * sz[child]`.
17. **Normalize indices in max width** — `normIdx = idx - firstIdx` per level. Prevents overflow.
18. **Column indexing**: `root=0`, `left=col-1`, `right=col+1`. Basis for top/bottom/vertical view.
19. **Serialize** = preorder + `"#"` for null; deserialize with index/queue consuming tokens.
20. **Build from pre+inorder**: first preorder = root; find in inorder hashmap; recurse left/right.

20 Most Important C++ Patterns

#	Pattern	C++ One-Liner
1	Height	<code>return !node ? 0 : 1 + max(height(node->left), height(node->right));</code>
2	Diameter	<code>maxD = max(maxD, L+R); return 1+max(L,R);</code>
3	Balanced	<code>if (abs(L-R)>1) return -1; return 1+max(L,R);</code>
4	Max Path Sum	<code>maxS = max(maxS, node->val + max(0,L) + max(0,R)); return node->val + max({0,L,R});</code>

5	LCA binary tree	if (!node node==p node==q) return node; both non-null return node
6	Validate BST	validate(node, LLONG_MIN, LLONG_MAX) with bounds narrowing
7	Kth smallest BST	inorder(left); if (++count==k) result=val; inorder(right);
8	BFS level loop	int sz=q.size(); for(int i=0;i<sz;i++) { ... }
9	Parent map + BFS	unordered_map<TreeNode*,TreeNode*> pm; buildParentMap(root,nullptr,pm);
10	Column map	map<int,int> cm; queue<pair<TreeNode*,int>> q; q.push({root,0});
11	Max width	long long norm = idx - firstIdx; push 2*norm and 2*norm+1
12	House Robber III	return {val+ISkip+rSkip, max(IR,IS)+max(rR,rS)};
13	Binary Lifting	up[v][j] = up[up[v][j-1]][j-1];
14	Euler Tour	tin[v]=timer++; dfs(children); tout[v]=timer++;
15	Serialize	preorder: push val then "#" for null
16	Build tree	root=pre[i++]; inMap[val]=idx; left uses [inL,mid-1], right [mid+1,inR]
17	Rerooting	dist[v] = dist[node] - sz[v] + (n - sz[v]);
18	Reverse inorder	right root left; if (++count==k) result=val;
19	Inorder Successor	while(root): if p->val<root->val: succ=root, root=left; else root=right
20	Path Sum III	int cnt = prefixCount[curSum - target]; prefixCount[curSum]++; ... --

20 Most Important Interview Observations

1. **"Any two nodes"** always a global variable problem. Never just root-to-leaf.

2. **Skewed BST worst case = $O(n)$** . Always mention alongside $O(\log n)$.
3. **BFS space = $O(n)$** for complete binary tree (last level has $n/2$ nodes).
4. **Don't assume LCA inputs exist** unless stated. Handle nullptr returns carefully.
5. **Path Sum III** is the prefix sum / hashmap trick on trees — same as “subarray sum = k ”.
6. **Morris Traversal** gives $O(1)$ space inorder — impressive to mention and implement.
7. **Inorder of general binary tree $\neq$ sorted**. Sorted = BST only.
8. **Clarify height definition** — edges (leaf=0) vs nodes (leaf=1). Do it at interview start.
9. **Postorder = natural deletion order** — must delete children before parent.
10. **Rerooting DP is $O(n)$** vs naive $O(n^2)$. Explain the optimization to impress.
11. **Tree DP = postorder DFS where return value IS the DP state**. Say this out loud.
12. **Right side view $\neq$ rightmost nodes** — if left subtree deeper, left nodes appear.
13. **Vertical order (LC 987) requires stable sort by (col, row, val)** — all three fields.
14. **Binary Lifting LCA**: if `up[u][j] == up[v][j]` do NOT lift. LCA is between.
15. **Complete Binary Tree node count** = $O(\log^2 n)$, not $O(n)$ — binary search on height.
16. **Iterative postorder** is tricky in C++ — use two-stack or reverse of modified preorder.
17. **Segment tree + Euler Tour** = $O(\log n)$ per subtree query with updates.
18. **In BST, inorder successor with right subtree** = `findMin(node->right)` . No parent needed.
19. **Diameter function returns HEIGHT, not diameter** — global holds the actual answer.
20. **When stuck**: Write brute force identify repeated sub-computation that's your DP state.

Ultra-Quick Pattern Cheat Card

LEVEL / WIDTH / SHORTEST	BFS (queue + <code>int sz=q.size()</code>)
BST / SORTED / KTH	INORDER (left root right)
SUBTREE AGGREGATE	POSTORDER (L,R computed before root)
CARRY INFO DOWN	PREORDER (state as parameter)
BEST ACROSS ALL NODES	TREE DP (global + postorder)
MANY LCA QUERIES	BINARY LIFTING (<code>up[v][j]</code> table)
SUBTREE AS RANGE	EULER TOUR (<code>tin/tout</code> + SegTree)
EVERY NODE AS ROOT	REROOTING (two DFS passes)

DISTANCE K / BURN

PARENT MAP + BFS

TOP / BOTTOM / VERTICAL

BFS + COLUMN MAP (`map<int,int>`)

Trees Master Revision Handbook (C++) — End

Built for SDE placements | Pattern-first | Interview-focused